PropOS CI/CD 流水线文档

> **版本**: v2.0
>
> **日期**: 2026-04-08
>
> **依据文档**: DEPLOYMENT v1.0 / TEST*PLAN v1.0* / *ARCH v1.4* / *DEV*ENV_SETUP v1.0
>
> **适用阶段**: Phase 1（单机部署 + GitHub Actions）

目录

一、流水线总览

二、分支策略

三、触发规则

四、Stage 1：代码质量检查

- 4.1 后端 Dart 检查
- 4.2 前端检查
- 4.3 自定义 Lint 规则

五、Stage 2：后端构建与测试

- 5.1 构建步骤
- 5.2 集成测试（需数据库）
- 5.3 API E2E 测试
- 5.4 覆盖率门控

六、Stage 3：前端构建与测试

- 6.1 构建步骤
- 6.2 构建制品

七、Stage 4：Docker 镜像构建

- 7.1 镜像构建
- 7.2 镜像命名规则

八、Stage 5：部署

- 8.1 Staging 自动部署
- 8.2 Production 手动确认部署
- 8.3 部署流程图

九、环境定义

- 环境变量差异

十、Secret 管理

- 10.1 Secret 清单

10.2 安全规则
十一、回滚策略
11.1 后端回滚
11.2 数据库回滚
11.3 前端静态文件回滚
11.4 回滚决策表
十二、流水线配置文件
12.1 CI 流水线 `.github/workflows/ci.yml`
12.2 夜间回归 `.github/workflows/nightly.yml`
十三、数据库迁移自动化
13.1 迁移脚本管理
13.2 迁移执行脚本
13.3 迁移安全规则
十四、制品管理
14.1 制品清单
14.2 版本号规范
十五、通知与告警
15.1 CI 通知规则
15.2 Webhook 配置
十六、常见问题
Q1: CI 中 `dart pub get` 超时怎么办?
Q2: 集成测试连不上 PostgreSQL?
Q3: Docker 镜像体积过大?
Q4: 如何跳过某个 Stage?
Q5: staging 和 production 数据库 schema 不一致?
Q6: 生产环境部署失败自动回滚后怎么办?

一、流水线总览

```
┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐
│ Stage 1  │──→│ Stage 2  │──→│ Stage 3  │──→│ Stage 4  │──→│ Stage 5  │
│ Lint &   │    │ Backend  │    │ Frontend │    │ Docker   │    │ Deploy   │
│ Analyze  │    │ Build &  │    │ Build &  │    │ Image    │    │          │
│          │    │ Test     │    │ Test     │    │ Build    │    │          │
└──────────┘    └──────────┘    └──────────┘    └──────────┘    └──────────┘
     │               │               │               │               │
dart analyze    Unit Tests      Vitest (app)    dart compile    docker compose
dart format     Integration     Vitest (admin)  docker build       up -d
```

```
eslint          API E2E          vite build        push registry
                                   (app + admin)
```

关键原则：

原则	说明
快速反馈	Lint + 单元测试控制在 5 分钟内完成，PR 可快速得到结果
逐级门控	前一 Stage 失败则后续 Stage 不执行，避免浪费资源
环境一致	构建与测试使用容器化环境，消除"我本地能跑"问题
制品不可变	同一 commit 只构建一次镜像，staging 和 production 使用同一制品
自动化优先	除生产部署需人工确认外，其余阶段全自动

二、分支策略

采用 **Trunk-Based Development**（适合一人全栈开发）：

```
main ──────●──────●──────●──────●────── (生产就绪)
           \    /  \    /
 feature/   ●─●     ●─●     feature/
 m1-assets         m2-contracts
```

分支	命名约定	用途	保护规则
`main`	—	生产就绪分支，始终可部署	禁止直接 push；必须通过 PR 合并；CI 全绿
`feature/*`	`feature/m1-assets`、`feature/be-auth`	功能开发分支	无保护；推送即触发 CI
`hotfix/*`	`hotfix/fix-wale-calc`	紧急修复	合并到 main 后自动触发部署

合并规则：

- PR 合并采用 **Squash Merge**，保持 main 历史线性清晰
- PR 标题格式：`[模块] 简述`，例如 `[M2] 完成合同状态机后端`
- PR 合并前必须：CI 全绿 + 无 conflict

三、触发规则

事件	触发 Stage	说明
Push to `feature/*`	Stage 1 → 2 → 3	代码质量 + 构建 + 测试
PR to `main`	Stage 1 → 2 → 3	同上，作为合并门控
Merge to `main`	Stage 1 → 2 → 3 → 4 → 5(staging)	全流水线，自动部署到 staging
Tag `v*` (如 `v1.0.0`)	Stage 4 → 5(production)	生产部署，需人工确认
手动触发	可选任意 Stage	用于回滚、重试、热修复部署
每日定时 00:30	Stage 2（集成测试子集）	夜间回归，捕获依赖变更引入的问题

四、Stage 1：代码质量检查

目标：在 2 分钟内完成代码风格和静态分析，提供最快反馈。

4.1 后端 Dart 检查

```
steps:
  - name: dart format --set-exit-if-changed .
    # 检查代码格式是否符合 Dart 标准，不一致则失败
  - name: dart analyze --fatal-infos
    # 静态分析，info 级别以上问题视为失败
  - name: dart pub deps --no-dev --style=compact
    # 输出依赖树，确认无意外间接依赖
```

4.2 前端检查

```
steps:
  - name: flutter analyze (flutter_app)
    run: cd flutter_app && flutter pub get && flutter analyze
  - name: npm run lint (admin)
    run: cd admin && npm ci && npm run lint
  - name: flutter test (flutter_app)
    run: cd flutter_app && flutter test
  - name: vue-tsc type check (admin)
    run: cd admin && npx vue-tsc --noEmit
```

4.3 自定义 Lint 规则

基于项目架构约束做强制检查（可用 shell 脚本或 ESLint 插件）：

检查项	命令	失败说明

Store 不直接写 fetch/axios	`grep -r "fetch(\	axios\." app/src/stores/ admin/src/stores/^
Page/Component 不含 HTTP 调用	`grep -rn "apiGet\	apiPost\
无硬编码 API 路径	`grep -rn '"/api/' app/src/pages/ app/src/stores/ admin/src/views/ admin/src/stores/`	统一使用 constants/api_paths
Controller 不直接返回 Response	`grep -r "return Response\." backend/lib/modules/`	Controller 只抛 AppException
SQL 无字符串拼接	`grep -rn '"\$' backend/lib/modules/*/repositories/`	防 SQL 注入
无 ORM 引入	`grep -rn "drift\	sqflite\

五、Stage 2：后端构建与测试

目标：编译后端 + 运行全部后端测试，10 分钟内完成。

5.1 构建步骤

```yaml
steps:
  - uses: dart-lang/setup-dart@v1
    with:
      sdk: "3.6"

  # 核心 Package 测试（零依赖，最快）
  - name: Test rent_escalation_engine
    run: cd packages/rent_escalation_engine && dart test --reporter=github

  - name: Test kpi_scorer
    run: cd packages/kpi_scorer && dart test --reporter=github

  # 后端依赖安装
  - name: Backend pub get
    run: cd backend && dart pub get

  # 代码生成（freezed / json_serializable）
  - name: Build runner
    run: cd backend && dart run build_runner build --delete-conflicting-outputs

  # 后端单元测试
  - name: Backend unit tests
    run: cd backend && dart test --reporter=github --coverage=coverage/

  # 覆盖率检查
  - name: Check coverage threshold
    run: |
      dart pub global activate coverage
```

```
      cd backend
      dart pub global run coverage:format_coverage \
        --lcov --in=coverage --out=coverage/lcov.info --report-on=lib/
      # 检查核心引擎覆盖率 ≥ 95%
```

5.2 集成测试（需数据库）

使用 GitHub Actions 的 service container 提供 PostgreSQL：

```
services:
  postgres:
    image: postgres:15-alpine
    env:
      POSTGRES_DB: propos_test
      POSTGRES_USER: test
      POSTGRES_PASSWORD: test
    ports:
      - 5432:5432
    options: >-
      --health-cmd pg_isready
      --health-interval 5s
      --health-timeout 3s
      --health-retries 5

steps:
  - name: Run migrations
    run: |
      for f in backend/migrations/*.sql; do
        psql -h localhost -U test -d propos_test -f "$f"
      done
    env:
      PGPASSWORD: test

  - name: Backend integration tests
    run: cd backend && dart test --tags=integration --reporter=github
    env:
      DATABASE_URL: postgres://test:test@localhost:5432/propos_test
      JWT_SECRET: ci-test-secret-minimum-32-characters-long
      JWT_EXPIRES_IN_HOURS: "1"
      FILE_STORAGE_PATH: /tmp/propos-test-uploads
      ENCRYPTION_KEY: "0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef"
      APP_PORT: "8080"
```

5.3 API E2E 测试

```
steps:
  - name: Start test server
    run: cd backend && dart run bin/server.dart &
    env:
      # 使用上述测试环境变量

  - name: Wait for server ready
```

```
    run: |
      for i in $(seq 1 30); do
        curl -sf http://localhost:8080/api/health && break
        sleep 1
      done

  - name: Run API E2E tests
    run: cd backend && dart test --tags=e2e --reporter=github
```

5.4 覆盖率门控

范围	最低覆盖率	失败行为
`rent_escalation_engine`	95%	CI 红灯
`kpi_scorer`	95%	CI 红灯
`backend/lib/modules/*/services/`	85%	CI 红灯
`backend/lib/modules/*/repositories/`	80%	CI 黄灯（警告）
总体	75%	CI 黄灯（警告）

六、Stage 3：前端构建与测试

目标：Flutter 端 + admin 端 Lint / 单元测试 / 构建，8 分钟内完成。

6.1 构建步骤

```
steps:
  - uses: subosito/flutter-action@v2
    with:
      flutter-version: "3.x"
      channel: stable

  - uses: actions/setup-node@v4
    with:
      node-version: "20"

  # Flutter 端
  - name: Install Flutter dependencies
    run: cd flutter_app && flutter pub get

  - name: Flutter analyze
    run: cd flutter_app && flutter analyze

  - name: Flutter unit tests
    run: cd flutter_app && flutter test --coverage
```

```
  - name: Flutter build (Web)
    run: cd flutter_app && flutter build web --release

  # Admin 端
  - name: Install admin dependencies
    run: cd admin && npm ci

  - name: Admin lint
    run: cd admin && npm run lint

  - name: Admin unit tests
    run: cd admin && npm run test -- --reporter=github --coverage

  - name: Admin build
    run: cd admin && npm run build
```

6.2 构建制品

```
  - name: Upload Flutter web artifact
    uses: actions/upload-artifact@v4
    with:
      name: flutter-web-${{ github.sha }}
      path: flutter_app/build/web/
      retention-days: 30

  - name: Upload admin artifact
    uses: actions/upload-artifact@v4
    with:
      name: admin-web-${{ github.sha }}
      path: admin/dist/
      retention-days: 30
```

七、Stage 4：Docker 镜像构建

目标：构建后端 Docker 镜像并推送到镜像仓库。仅在 `main` 分支合并或打 tag 时执行。

7.1 镜像构建

```
steps:
  - name: Set image tag
    run: |
      if [[ "$GITHUB_REF" == refs/tags/v* ]]; then
        echo "IMAGE_TAG=${GITHUB_REF#refs/tags/}" >> $GITHUB_ENV
      else
        echo "IMAGE_TAG=sha-${GITHUB_SHA::8}" >> $GITHUB_ENV
      fi

  - name: Build backend image
    run: |
      docker build \
```

```
        -f backend/Dockerfile \
        -t propos-backend:${{ env.IMAGE_TAG }} \
        -t propos-backend:latest \
        .

  - name: Scan image for vulnerabilities
    uses: aquasecurity/trivy-action@master
    with:
      image-ref: propos-backend:${{ env.IMAGE_TAG }}
      format: table
      exit-code: 1
      severity: CRITICAL,HIGH

  - name: Push to registry
    run: |
      docker tag propos-backend:${{ env.IMAGE_TAG }} $REGISTRY/propos-backend:${{ env.IMAGE_TAG
}}
      docker push $REGISTRY/propos-backend:${{ env.IMAGE_TAG }}
```

7.2 镜像命名规则

场景	镜像 Tag	示例
main 合并	`sha-<commit前8位>`	`sha-a1b2c3d4`
版本发布	`v<语义化版本>`	`v1.0.0`
最新 main	`latest`	`latest`

八、Stage 5：部署

8.1 Staging 自动部署

main 合并后自动部署到 staging 环境：

```
deploy-staging:
  needs: [build-image]
  if: github.ref == 'refs/heads/main'
  environment:
    name: staging
    url: https://staging.propos.example.com
  steps:
    - name: SSH deploy
      uses: appleboy/ssh-action@master
      with:
        host: ${{ secrets.STAGING_HOST }}
        username: ${{ secrets.DEPLOY_USER }}
        key: ${{ secrets.DEPLOY_SSH_KEY }}
        script: |
```

```
          cd /opt/propos

          # 拉取最新镜像
          docker pull $REGISTRY/propos-backend:sha-${{ github.sha }}

          # 更新 .env 中的镜像 tag
          sed -i "s|BACKEND_IMAGE=.*|BACKEND_IMAGE=$REGISTRY/propos-backend:sha-$
{{ github.sha }}|" .env

          # 部署前备份数据库
          docker compose exec -T postgres pg_dump -U propos propos > /backups/pre-deploy-$(date
+%Y%m%d%H%M%S).sql

          # 执行数据库迁移
          bash scripts/run_migrations.sh

          # 滚动更新后端
          docker compose up -d backend

          # 更新前端静态文件（app H5 + admin）
          # (从 CI 制品下载或通过 rsync 同步)

          # 健康检查
          for i in $(seq 1 30); do
            curl -sf https://staging.propos.example.com/api/health && exit 0
            sleep 2
          done
          echo "Health check failed" && exit 1
```

8.2 Production 手动确认部署

打 tag 触发，需要在 GitHub UI 确认后才执行：

```
deploy-production:
  needs: [build-image]
  if: startsWith(github.ref, 'refs/tags/v')
  environment:
    name: production
    url: https://propos.example.com
  steps:
    - name: SSH deploy
      uses: appleboy/ssh-action@master
      with:
        host: ${{ secrets.PRODUCTION_HOST }}
        username: ${{ secrets.DEPLOY_USER }}
        key: ${{ secrets.DEPLOY_SSH_KEY }}
        script: |
          cd /opt/propos

          # 完整数据库备份
          docker compose exec -T postgres pg_dump -U propos propos | gzip > /backups/prod-$
(date +%Y%m%d%H%M%S).sql.gz
```

```
          # 记录当前运行版本（用于回滚）
          docker inspect --format='{{index .Config.Image}}' propos-backend > /opt/propos/.last-
good-image

          # 拉取发布版本镜像
          docker pull $REGISTRY/propos-backend:${{ github.ref_name }}
          sed -i "s|BACKEND_IMAGE=.*|BACKEND_IMAGE=$REGISTRY/propos-backend:${{ github.ref_name
}}|" .env

          # 执行数据库迁移
          bash scripts/run_migrations.sh

          # 滚动更新
          docker compose up -d backend

          # 健康检查（生产环境等待更长时间）
          for i in $(seq 1 60); do
            curl -sf https://propos.example.com/api/health && exit 0
            sleep 2
          done
          echo "Health check failed — triggering rollback"
          bash scripts/rollback.sh
          exit 1
```

8.3 部署流程图

```
                    ┌──────────┐
                    │ PR 合并  │
                    │ to main  │
                    └──────────┘
                         │
                    ┌────▼─────┐
                    │ CI 全流程│
                    │ Stage1~4 │
                    └──────────┘
                         │ 自动
                    ┌────▼─────┐
                    │  Staging │
                    │ 自动部署 │
                    └──────────┘
                         │ 验证通过
                    ┌────▼─────┐
                    │ 打 Tag   │
                    │ v1.x.x   │
                    └──────────┘
                         │ 人工确认
                    ┌────▼──────┐
                    │ Production │
                    │ 手动确认部署 │
                    └────────────┘
```

九、环境定义

环境	域名	用途	部署方式	数据库
local	`localhost:8080`	开发调试	`dart run bin/server.dart`	本地 PostgreSQL
staging	`staging.propos.example.com`	集成验证、UAT	CI 自动部署	独立 PostgreSQL（可重置）
production	`propos.example.com`	正式运行	Tag 触发 + 人工确认	生产 PostgreSQL（每日备份）

环境变量差异

变量	local	staging	production
`LOG_LEVEL`	`debug`	`info`	`warn`
`CORS_ORIGINS`	`*`	`https://staging.propos.example.com`	`https://propos.example.com`
`JWT_EXPIRES_IN_HOURS`	`24`	`24`	`12`
`MAX_UPLOAD_SIZE_MB`	`50`	`50`	`50`

十、Secret 管理

所有敏感信息通过 **GitHub Actions Secrets**（环境级别）管理，禁止提交到代码仓库。

10.1 Secret 清单

Secret 名称	范围	说明
`DB_PASSWORD`	staging / production	PostgreSQL 密码
`JWT_SECRET`	staging / production	JWT 签名密钥（≥32 位）
`ENCRYPTION_KEY`	staging / production	AES-256 加密密钥（32 字节 hex）
`DEPLOY_SSH_KEY`	staging / production	部署用 SSH 私钥
`STAGING_HOST`	staging	Staging 服务器 IP
`PRODUCTION_HOST`	production	生产服务器 IP
`DEPLOY_USER`	staging / production	SSH 部署用户名
`REGISTRY_TOKEN`	全局	镜像仓库认证 Token

10.2 安全规则

1. Staging 和 Production 使用不同的 `JWT_SECRET` 和 `ENCRYPTION_KEY`，防止跨环境 Token 重放
2. `DEPLOY_SSH_KEY` 对应的服务器用户仅有 Docker 操作权限，不可 `sudo`
3. 每季度轮换 `JWT_SECRET` 和 `ENCRYPTION_KEY`，轮换时需执行 token 失效 + 数据重加密脚本
4. CI 日志中通过 `::add-mask::` 屏蔽所有 secret 值

十一、回滚策略

11.1 后端回滚

```
#!/bin/bash
# scripts/rollback.sh — 后端服务回滚到上一个已知正常版本

set -euo pipefail

LAST_IMAGE=$(cat /opt/propos/.last-good-image)
echo "Rolling back to: $LAST_IMAGE"

cd /opt/propos
sed -i "s|BACKEND_IMAGE=.*|BACKEND_IMAGE=$LAST_IMAGE|" .env
docker compose up -d backend

# 等待健康检查
for i in $(seq 1 30); do
  curl -sf https://propos.example.com/api/health && echo "Rollback successful" && exit 0
  sleep 2
done

echo "CRITICAL: Rollback also failed!" >&2
```

```
exit 1
```

11.2 数据库回滚

```
#!/bin/bash
# scripts/db_rollback.sh — 数据库回滚到指定备份

set -euo pipefail

BACKUP_FILE=$1

if [[ -z "$BACKUP_FILE" ]]; then
  echo "Usage: $0 <backup_file.sql.gz>"
  echo "Available backups:"
  ls -lt /backups/prod-*.sql.gz | head -10
  exit 1
fi

echo "WARNING: This will overwrite the current database!"
read -p "Type 'yes' to continue: " confirm
[[ "$confirm" != "yes" ]] && echo "Aborted" && exit 0

cd /opt/propos
docker compose exec -T postgres dropdb -U propos propos
docker compose exec -T postgres createdb -U propos propos
gunzip -c "$BACKUP_FILE" | docker compose exec -T postgres psql -U propos propos

echo "Database restored from $BACKUP_FILE"
```

11.3 前端静态文件回滚

前端为静态文件（app H5 + admin），每次部署前保存上一版本：

```
# 部署时自动执行
cp -r /opt/propos/deploy/nginx/html /opt/propos/deploy/nginx/html.bak

# 回滚
rm -rf /opt/propos/deploy/nginx/html
mv /opt/propos/deploy/nginx/html.bak /opt/propos/deploy/nginx/html
docker compose restart nginx
```

11.4 回滚决策表

问题类型	回滚方式	预期恢复时间
后端 API 500	镜像回滚（`rollback.sh`）	< 2 分钟
数据库迁移错误	反向迁移脚本 + 镜像回滚	< 10 分钟
数据损坏	数据库备份恢复（`db_rollback.sh`）	< 15 分钟
前端 UI 问题	静态文件回滚	< 1 分钟
全栈问题	全量回滚（镜像 + DB + Web）	< 20 分钟

十二、流水线配置文件

12.1 CI 流水线 `.github/workflows/ci.yml`

```yaml
name: PropOS CI

on:
  push:
    branches: [main, "feature/**", "hotfix/**"]
  pull_request:
    branches: [main]

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

env:
  DART_SDK_VERSION: "3.6"
  NODE_VERSION: "20"

jobs:
  # ────────────────────────────────────────────
  # Stage 1: Lint & Analyze
  # ────────────────────────────────────────────
  lint:
    name: "Stage 1: Lint & Analyze"
    runs-on: ubuntu-latest
    timeout-minutes: 5
    steps:
      - uses: actions/checkout@v4

      - uses: dart-lang/setup-dart@v1
        with:
          sdk: ${{ env.DART_SDK_VERSION }}

      - uses: actions/setup-node@v4
```

```
      with:
        node-version: ${{ env.NODE_VERSION }}

    # 后端Lint
    - name: Backend format check
      run: dart format --set-exit-if-changed backend/

    - name: Backend analyze
      run: |
        cd backend && dart pub get
        dart analyze --fatal-infos

    # Package Lint
    - name: Package format check
      run: |
        dart format --set-exit-if-changed packages/rent_escalation_engine/
        dart format --set-exit-if-changed packages/kpi_scorer/

    # 前端Lint（flutter_app）
    - name: Flutter analyze
      run: cd flutter_app && flutter pub get && flutter analyze

    # 前端Lint（admin）
    - name: Admin lint
      run: cd admin && npm ci && npm run lint

    # 架构约束检查
    - name: Architecture constraints
      run: |
        echo "--- Checking stores do not use fetch/axios directly ---"
        ! grep -rn "fetch(\|axios\." flutter_app/lib/ admin/src/stores/ 2>/dev/null || \
          (echo "FAIL: store imports fetch/axios directly" && exit 1)

        echo "--- Checking no ORM ---"
        ! grep -rn "drift\|sqflite\|floor" backend/pubspec.yaml 2>/dev/null || \
          (echo "FAIL: ORM detected in backend" && exit 1)

        echo "--- Architecture checks passed ---"

# ──────────────────────────────────────────────
# Stage 2: Backend Build & Test
# ──────────────────────────────────────────────
backend-test:
  name: "Stage 2: Backend Tests"
  needs: lint
  runs-on: ubuntu-latest
  timeout-minutes: 15
  services:
    postgres:
      image: postgres:15-alpine
      env:
        POSTGRES_DB: propos_test
        POSTGRES_USER: test
```

```
          POSTGRES_PASSWORD: test_ci_password
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 5s
          --health-timeout 3s
          --health-retries 10
    steps:
      - uses: actions/checkout@v4

      - uses: dart-lang/setup-dart@v1
        with:
          sdk: ${{ env.DART_SDK_VERSION }}

      # 缓存 pub 依赖
      - uses: actions/cache@v4
        with:
          path: ~/.pub-cache
          key: pub-${{ hashFiles('**/pubspec.lock') }}
          restore-keys: pub-

      # 核心 Package 单元测试（最快，最先执行）
      - name: Test rent_escalation_engine
        run: |
          cd packages/rent_escalation_engine
          dart pub get
          dart test --reporter=github --coverage=coverage/

      - name: Test kpi_scorer
        run: |
          cd packages/kpi_scorer
          dart pub get
          dart test --reporter=github --coverage=coverage/

      # 后端构建
      - name: Backend pub get & build_runner
        run: |
          cd backend
          dart pub get
          dart run build_runner build --delete-conflicting-outputs

      # 后端单元测试
      - name: Backend unit tests
        run: cd backend && dart test --exclude-tags=integration,e2e --reporter=github --
coverage=coverage/

      # 数据库迁移
      - name: Run database migrations
        run: |
          for f in $(ls backend/migrations/*.sql 2>/dev/null | sort); do
            echo "Running: $f"
            PGPASSWORD=test_ci_password psql -h localhost -U test -d propos_test -f "$f"
```

```
          done

      # 后端集成测试
      - name: Backend integration tests
        run: cd backend && dart test --tags=integration --reporter=github
        env:
          DATABASE_URL: postgres://test:test_ci_password@localhost:5432/propos_test
          JWT_SECRET: ci-test-secret-key-must-be-at-least-32-chars
          JWT_EXPIRES_IN_HOURS: "1"
          FILE_STORAGE_PATH: /tmp/propos-test-uploads
          ENCRYPTION_KEY: "0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef"
          APP_PORT: "8080"

      # API E2E 测试
      - name: Start test server
        run: |
          cd backend && dart run bin/server.dart &
          for i in $(seq 1 30); do curl -sf http://localhost:8080/api/health && break; sleep 1;
done
        env:
          DATABASE_URL: postgres://test:test_ci_password@localhost:5432/propos_test
          JWT_SECRET: ci-test-secret-key-must-be-at-least-32-chars
          JWT_EXPIRES_IN_HOURS: "1"
          FILE_STORAGE_PATH: /tmp/propos-test-uploads
          ENCRYPTION_KEY: "0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef"
          APP_PORT: "8080"

      - name: Backend E2E tests
        run: cd backend && dart test --tags=e2e --reporter=github

      # 覆盖率收集
      - name: Collect coverage
        run: |
          dart pub global activate coverage
          cd backend
          dart pub global run coverage:format_coverage \
            --lcov --in=coverage --out=coverage/lcov.info --report-on=lib/

      - name: Upload coverage
        uses: actions/upload-artifact@v4
        with:
          name: backend-coverage
          path: backend/coverage/lcov.info

  # ────────────────────────────────────────────────
  # Stage 3: Frontend Build & Test
  # ────────────────────────────────────────────────
  frontend-test:
    name: "Stage 3: Frontend Tests"
    needs: lint
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
```

```yaml
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}

      - uses: actions/cache@v4
        with:
          path: |
            flutter_app/.dart_tool
            admin/node_modules
          key: deps-${{ hashFiles('flutter_app/pubspec.lock', 'admin/package-lock.json') }}
          restore-keys: deps-

      # Flutter 端
      - name: Install Flutter dependencies
        run: cd flutter_app && flutter pub get

      - name: Flutter unit tests with coverage
        run: cd flutter_app && flutter test --coverage

      - name: Build Flutter (Web)
        run: cd flutter_app && flutter build web --release

      # Admin 端
      - name: Install admin dependencies
        run: cd admin && npm ci

      - name: Admin unit tests with coverage
        run: cd admin && npm run test -- --reporter=github --coverage

      - name: Build admin
        run: cd admin && npm run build

      - name: Upload Flutter web build
        uses: actions/upload-artifact@v4
        with:
          name: flutter-web-${{ github.sha }}
          path: flutter_app/build/web/
          retention-days: 30

      - name: Upload admin build
        uses: actions/upload-artifact@v4
        with:
          name: admin-web-${{ github.sha }}
          path: admin/dist/
          retention-days: 30

      - name: Upload coverage
        uses: actions/upload-artifact@v4
        with:
          name: frontend-coverage
          path: |
```

```
          flutter_app/coverage/
          admin/coverage/

    - name: Build Flutter web
      run: cd flutter_app && flutter build web --release

    - name: Upload Flutter web build
      uses: actions/upload-artifact@v4
      with:
        name: flutter-web-${{ github.sha }}
        path: flutter_app/build/web/
        retention-days: 30

    - name: Upload admin build
      uses: actions/upload-artifact@v4
      with:
        name: admin-web-${{ github.sha }}
        path: admin/dist/
        retention-days: 30

    - name: Upload coverage
      uses: actions/upload-artifact@v4
      with:
        name: frontend-coverage
        path: |
          flutter_app/coverage/
          admin/coverage/

# ──────────────────────────────────────────────
# Stage 4: Docker Image Build
# ──────────────────────────────────────────────
build-image:
  name: "Stage 4: Docker Image"
  needs: [backend-test, frontend-test]
  if: github.ref == 'refs/heads/main' || startsWith(github.ref, 'refs/tags/v')
  runs-on: ubuntu-latest
  timeout-minutes: 15
  steps:
    - uses: actions/checkout@v4

    - name: Set image tag
      run: |
        if [[ "$GITHUB_REF" == refs/tags/v* ]]; then
          echo "IMAGE_TAG=${GITHUB_REF#refs/tags/}" >> $GITHUB_ENV
        else
          echo "IMAGE_TAG=sha-${GITHUB_SHA::8}" >> $GITHUB_ENV
        fi

    - name: Build Docker image
      run: |
        docker build \
          -f backend/Dockerfile \
          -t propos-backend:${{ env.IMAGE_TAG }} \
          -t propos-backend:latest \
```

```
          --label "org.opencontainers.image.revision=${{ github.sha }}" \
          --label "org.opencontainers.image.created=$(date -u +%Y-%m-%dT%H:%M:%SZ)" \
          .

    - name: Scan for vulnerabilities
      uses: aquasecurity/trivy-action@master
      with:
        image-ref: propos-backend:${{ env.IMAGE_TAG }}
        format: table
        exit-code: 1
        severity: CRITICAL,HIGH

    - name: Save image as artifact
      run: docker save propos-backend:${{ env.IMAGE_TAG }} | gzip > /tmp/backend-image.tar.gz

    - name: Upload image artifact
      uses: actions/upload-artifact@v4
      with:
        name: backend-image-${{ env.IMAGE_TAG }}
        path: /tmp/backend-image.tar.gz
        retention-days: 30

# ──────────────────────────────────────────────
# Stage 5a: Deploy to Staging
# ──────────────────────────────────────────────
deploy-staging:
  name: "Stage 5: Deploy Staging"
  needs: build-image
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  environment:
    name: staging
    url: https://staging.propos.example.com
  steps:
    - uses: actions/checkout@v4

    - name: Download web builds
      uses: actions/download-artifact@v4
      with:
        pattern: '*-web-*'
        path: web-dist/

    - name: Deploy to staging server
      uses: appleboy/ssh-action@master
      with:
        host: ${{ secrets.STAGING_HOST }}
        username: ${{ secrets.DEPLOY_USER }}
        key: ${{ secrets.DEPLOY_SSH_KEY }}
        script: |
          set -euo pipefail
          cd /opt/propos

          # 备份数据库
```

```
            docker compose exec -T postgres pg_dump -U propos propos > /backups/staging-pre-$
(date +%Y%m%d%H%M%S).sql

            # 更新镜像
            docker compose pull backend || true
            docker compose up -d backend

            # 健康检查
            for i in $(seq 1 30); do
              curl -sf http://localhost:8080/api/health && exit 0
              sleep 2
            done
            echo "Staging deploy health check failed" && exit 1

  # ────────────────────────────────────────────
  # Stage 5b: Deploy to Production
  # ────────────────────────────────────────────
  deploy-production:
    name: "Stage 5: Deploy Production"
    needs: build-image
    if: startsWith(github.ref, 'refs/tags/v')
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://propos.example.com
    steps:
      - uses: actions/checkout@v4

      - name: Deploy to production server
        uses: appleboy/ssh-action@master
        with:
          host: ${{ secrets.PRODUCTION_HOST }}
          username: ${{ secrets.DEPLOY_USER }}
          key: ${{ secrets.DEPLOY_SSH_KEY }}
          script: |
            set -euo pipefail
            cd /opt/propos

            # 完整备份
            docker compose exec -T postgres pg_dump -U propos propos | gzip > /backups/prod-
pre-$(date +%Y%m%d%H%M%S).sql.gz

            # 记录当前版本用于回滚
            docker inspect --format='{{.Config.Image}}' propos-backend > .last-good-image 2>/
dev/null || true

            # 更新
            docker compose pull backend || true
            docker compose up -d backend

            # 健康检查
            for i in $(seq 1 60); do
```

```
              curl -sf https://propos.example.com/api/health && echo "Production deploy
successful" && exit 0
              sleep 2
            done

            echo "CRITICAL: Production health check failed — rolling back"
            bash scripts/rollback.sh
            exit 1
```

12.2 夜间回归 `.github/workflows/nightly.yml`

```
name: Nightly Regression

on:
  schedule:
    - cron: "30 16 * * *"  # UTC 16:30 = 北京时间 00:30
  workflow_dispatch:

jobs:
  nightly:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    services:
      postgres:
        image: postgres:15-alpine
        env:
          POSTGRES_DB: propos_test
          POSTGRES_USER: test
          POSTGRES_PASSWORD: test_ci_password
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 5s
          --health-timeout 3s
          --health-retries 10
    steps:
      - uses: actions/checkout@v4
      - uses: dart-lang/setup-dart@v1
        with:
          sdk: "3.6"
      # 全量后端测试
      - name: Full backend test suite
        run: |
          cd backend && dart pub get
          dart run build_runner build --delete-conflicting-outputs
          dart test --reporter=github --coverage=coverage/
        env:
          DATABASE_URL: postgres://test:test_ci_password@localhost:5432/propos_test
          JWT_SECRET: ci-test-secret-key-must-be-at-least-32-chars
          JWT_EXPIRES_IN_HOURS: "1"
          FILE_STORAGE_PATH: /tmp/propos-test-uploads
          ENCRYPTION_KEY: "0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef"
```

```
          APP_PORT: "8080"

      # 全量前端测试
      - name: Full frontend test suite
        run: |
          cd flutter_app && flutter pub get && flutter test
          cd ../admin && npm ci && npm run test
```

十三、数据库迁移自动化

13.1 迁移脚本管理

```
backend/migrations/
├── 001_create_enums.sql
├── 002_create_users_and_audit.sql
├── 003_create_assets.sql
├── ...
└── 017_create_kpi_targets_and_appeals.sql
```

13.2 迁移执行脚本

```
#!/bin/bash
# scripts/run_migrations.sh — 自动执行未运行的迁移

set -euo pipefail

DB_CONTAINER=${DB_CONTAINER:-propos-db}

# 确保迁移记录表存在
docker exec -i "$DB_CONTAINER" psql -U propos -d propos <<SQL
CREATE TABLE IF NOT EXISTS schema_migrations (
  version VARCHAR(10) PRIMARY KEY,
  filename VARCHAR(255) NOT NULL,
  applied_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
SQL

# 遍历迁移文件，跳过已执行的
for f in $(ls backend/migrations/*.sql 2>/dev/null | sort); do
  VERSION=$(basename "$f" | cut -d_ -f1)
  FILENAME=$(basename "$f")

  APPLIED=$(docker exec -i "$DB_CONTAINER" psql -U propos -d propos -tA \
    -c "SELECT COUNT(*) FROM schema_migrations WHERE version = '$VERSION'")

  if [[ "$APPLIED" -eq 0 ]]; then
    echo "==> Applying migration: $FILENAME"
    docker exec -i "$DB_CONTAINER" psql -U propos -d propos < "$f"
    docker exec -i "$DB_CONTAINER" psql -U propos -d propos \
```

```
      -c "INSERT INTO schema_migrations (version, filename) VALUES ('$VERSION', '$FILENAME')"
    echo "    Done: $FILENAME"
  else
    echo "    Skip (already applied): $FILENAME"
  fi
done

echo "All migrations complete."
```

13.3 迁移安全规则

规则	说明
只进不退	迁移文件一旦合入 main，不可修改或删除
向后兼容	新迁移不能破坏当前运行的代码（先加列，后续版本再移除旧列）
事务包裹	每个迁移文件内容用 `BEGIN; ... COMMIT;` 包裹
CI 验证	集成测试阶段从空库执行全部迁移，确保可重放
备份先行	生产执行迁移前必须有当日数据库备份

十四、制品管理

14.1 制品清单

制品	存储位置	保留策略
后端 Docker 镜像	Docker Registry	最近 30 个 tag
前端构建产物（app H5 + admin）	GitHub Actions Artifacts	30 天
测试覆盖率报告	GitHub Actions Artifacts	30 天
数据库备份	服务器 `/backups/`	Staging: 7 天 / Production: 90 天
迁移 SQL 文件	Git 仓库 `backend/migrations/`	永久（随代码版本控制）

14.2 版本号规范

采用 语义化版本 `vMAJOR.MINOR.PATCH` ：

组件	递增规则	示例
MAJOR	不兼容 API 变更或数据库 breaking change	`v2.0.0`
MINOR	新功能（向后兼容）	`v1.1.0` （完成 M1 资产模块）

PATCH	Bug 修复	`v1.0.1`

Phase 1 版本规划：

里程碑	Tag	说明
L1 主数据可用	`v0.1.0`	资产台账 + CAD + 导入
L2 业财闭环	`v0.2.0`	合同 + 账单 + NOI
L3 运营闭环	`v0.3.0`	工单 + 二房东
L4 上线就绪	`v1.0.0`	安全 + 审计 + 验收

十五、通知与告警

15.1 CI 通知规则

事件	通知方式	接收人
CI 失败（任意 Stage）	GitHub PR 状态检查标红	PR 作者
main 分支 CI 失败	钉钉/企业微信 Webhook	全团队
Staging 部署成功	GitHub Deployment 状态	—
Staging 部署失败	钉钉/企业微信 + GitHub Issue	全团队
Production 部署成功	钉钉/企业微信	全团队
Production 部署失败 + 自动回滚	钉钉/企业微信 + GitHub Issue（P0）	全团队
夜间回归失败	钉钉/企业微信	全团队
镜像漏洞扫描 CRITICAL	GitHub Security Alert	全团队

15.2 Webhook 配置

```
# 在 workflow 中添加通知步骤
- name: Notify on failure
  if: failure()
  run: |
    curl -X POST "${{ secrets.WEBHOOK_URL }}" \
      -H "Content-Type: application/json" \
      -d '{
        "msgtype": "markdown",
        "markdown": {
          "title": "PropOS CI 失败",
```

```
          "text": "### ❌ CI 失败\n- **分支**: ${{ github.ref_name }}\n- **提交**: ${{ github.sha
}}\n- **触发**: ${{ github.actor }}\n- [查看详情](${{ github.server_url }}/$
{{ github.repository }}/actions/runs/${{ github.run_id }})"
        }
      }'
```

十六、常见问题

Q1: CI 中 `dart pub get` 超时怎么办?

启用 `actions/cache` 缓存 `~/.pub-cache`，避免每次全量下载。若仍超时，检查是否有 Git 依赖需要 SSH 认证。

Q2: 集成测试连不上 PostgreSQL?

确认 service container 的 health check 通过后才执行测试。GitHub Actions 的 service container 端口映射到 `localhost`。

Q3: Docker 镜像体积过大?

当前 Dart AOT 编译后镜像基于 `debian:bookworm-slim`，约 30-50MB。如进一步优化可改用 `scratch` 基础镜像 + 静态编译。

Q4: 如何跳过某个 Stage?

在 commit message 中加入 `[skip ci]` 跳过整个流水线；加入 `[skip e2e]` 需自定义条件判断，不建议常态跳过。

Q5: staging 和 production 数据库 schema 不一致?

`run_migrations.sh` 基于 `schema_migrations` 表记录执行状态，确保两个环境执行相同的迁移序列。如出现不一致，先在 staging 验证迁移脚本，再部署 production。

Q6: 生产环境部署失败自动回滚后怎么办?

1. 检查 GitHub Actions 日志定位失败原因
2. 确认回滚是否成功（访问 `/api/health`）
3. 修复代码后重新走 PR → merge → staging 验证 → tag 发布流程
4. 禁止直接 SSH 到服务器手动修复代码